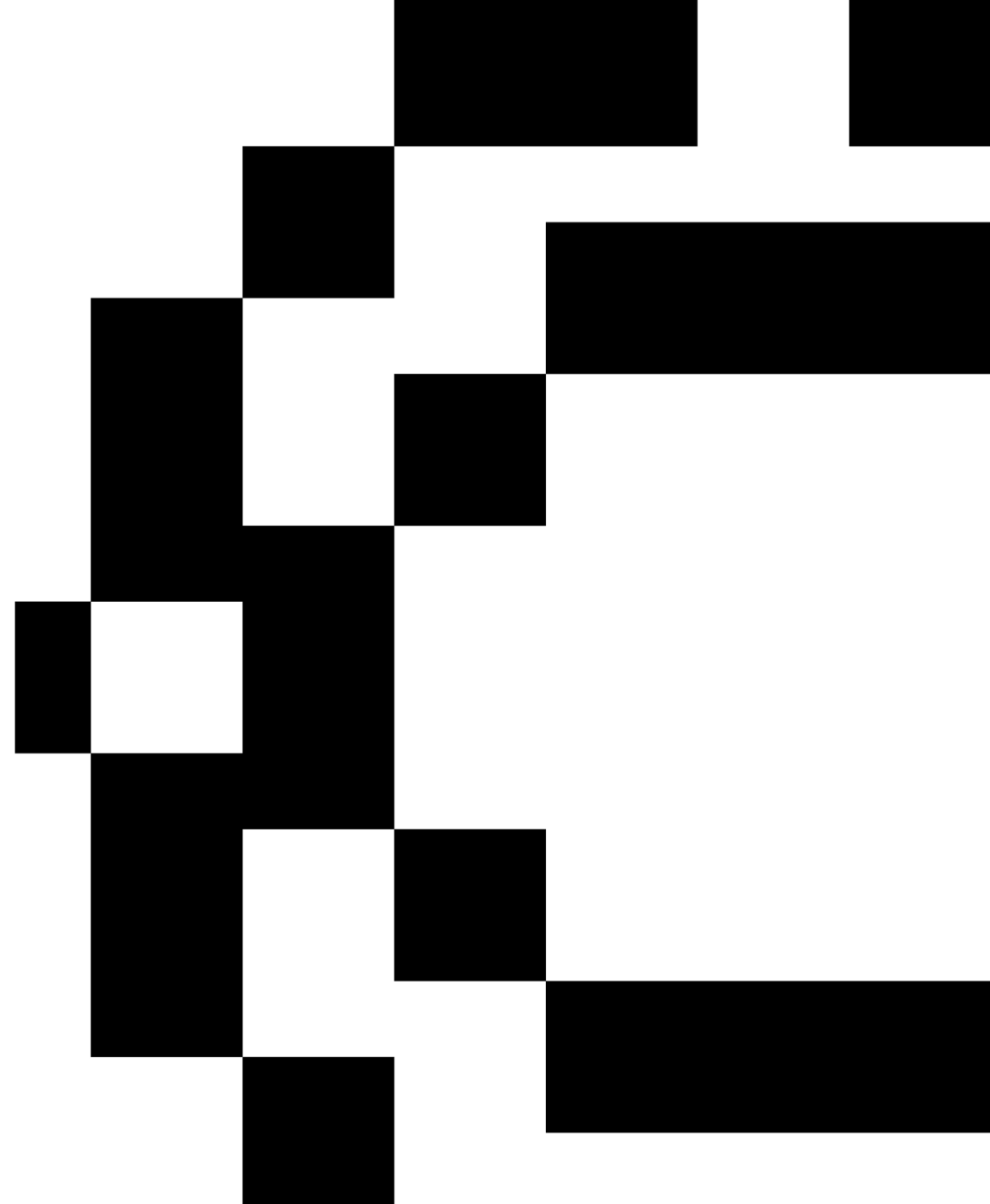


N-Grams, TF-IDF & Distance metrics

Text Mining 2025-2026

Bruno Jardim

bjardim@novaims.unl.pt





01. N-Grams

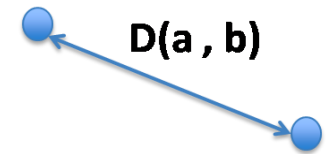
Recall Problems with Bag of Words

N-grams

Now is the time for
all good man to
come to the aid of
their party.

The quick brown fox
jumped over the
lazy dog's back.

$$D(a, b) = \sqrt{\sum_{i=1}^n (b_i - a_i)^2}$$



	aid	all	back	brown	come	dog	fox	good	jump	lazy	men	now	over	party	quick	their	Time
Doc1	0	0	1	1	0	1	1	0	1	1	0	0	1	0	1	0	0
Doc2	1	1	0	0	1	0	0	1	0	0	1	1	0	1	0	1	1

1. Curse of dimensionality
2. No semantic relationships
(**Word order** is discarded)
3. All words have the same importance

1. Curse of dimensionality
2. No semantic relationships
(Word order is discarded)
3. All words have the same importance

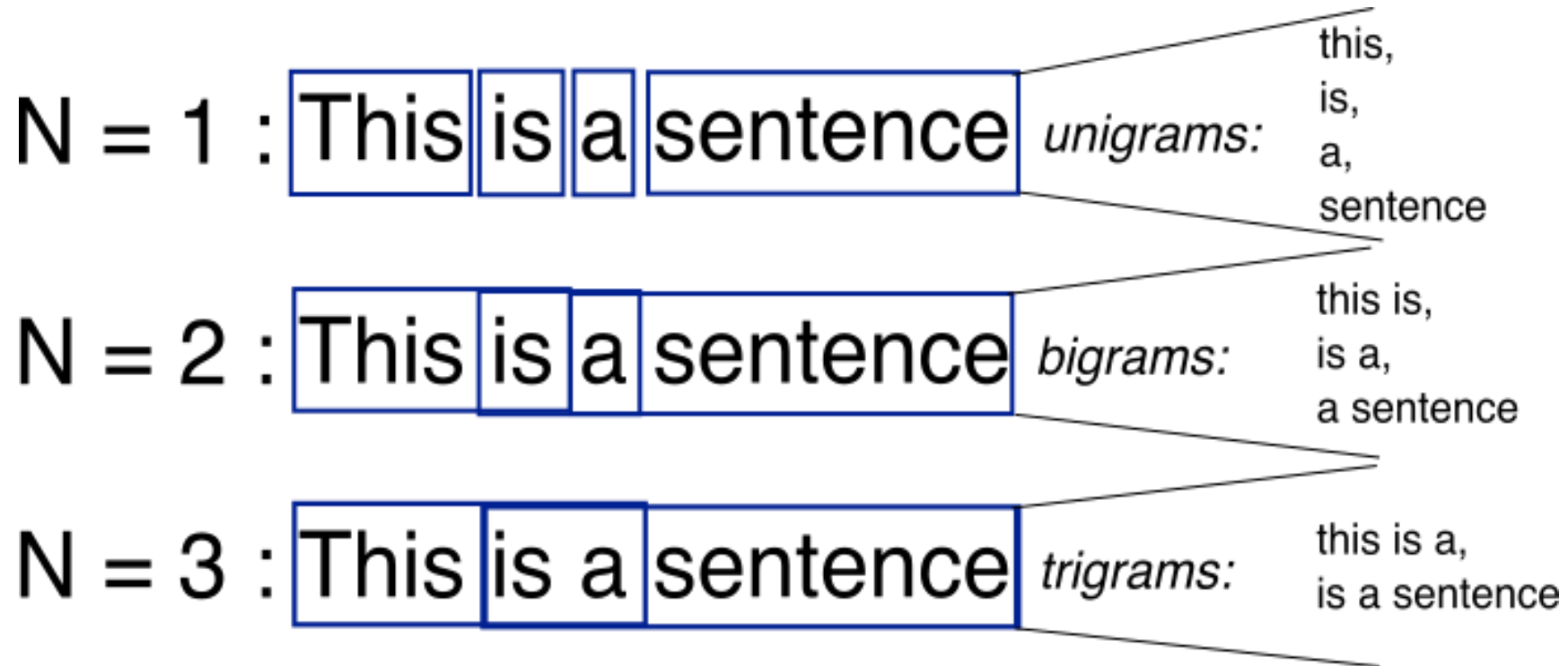
Definition:

An n-gram is a **contiguous sequence of n items** from a given sample (text or speech).

- Although N-grams can be sequences of anything (characters, words, lemmas, etc.) we will see mostly **sequences of words**.

What are N-grams

N-grams



The BoW can also be done with N-grams (ex: 2-grams):

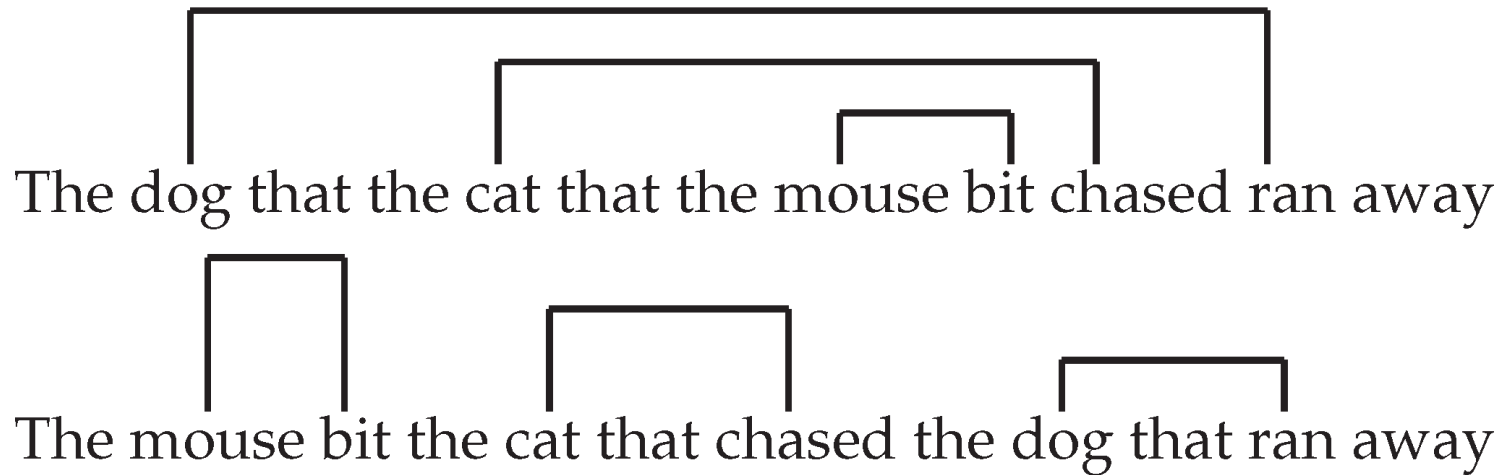
Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “ The cat is on the table”

	(The, dog)	(dog, is)	(is, on)	(on, the)	(the, table)	(The, cat)	(cat, is)
Term Frequency d1	1	1	1	1	1	0	0
Term Frequency d2	0	0	1	1	1	1	1

N-Gram size:

Sometimes, the occurrence of one word, depends on another word that came way before in text.



Data sparseness:

One of the major drawbacks with N-grams is that they **don't deal well with long distance dependencies**.

E.g:

"Gollum loves his precious" could be a frequent 4-gram in Lord of the Rings, but for some reason, our text might only contain things such as: "Gollum loves in a very sick way precious", thus the initial 4-gram will never be found.

Typically **when we increase N** our **results became sparse** (aka we get a lot of zeros in the BoW).



02.

TF-IDF

1. Curse of dimensionality
2. No semantic relationships
(Word order is discarded)
- 3. All words have the same importance**

All words have the same importance...

Possible solution:

- Count the number of times a word appears.

All words have the same importance...

Possible solution:

- Count the number of times a word appears.

**Unfortunately, words that appear most typically
are not the most important ones...**

Term Frequency – Inverse Document Frequency (TF-IDF)

$$\text{TF-IDF} = \text{TF}(t, d) * \text{IDF}(t, D)$$

$$\text{TF}(t, d) = \text{raw count of } t \text{ in } d$$

$$\text{IDF}(t, D) = \log\left(\frac{|D|}{n_t}\right)$$

n_t = number of documents where the term appears

Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “ The cat is on the table”

TF(“word”, d) – Count “word” in document **d**

IDF(“word”, D) – $\log(|D|/n_t)$, where **|D|** is the number of documents in the corpus (in this example, we have 2 documents) and **n_t** is the number of documents **n** with the word **t**

How does TF-IDF work?

TF-IDF

Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “The cat is on the table”

$TF(t, d) = \text{raw count of } t \text{ in } d$

	The	dog	is	on	table	cat
Term Frequency d1	2	1	1	1	1	0
Term Frequency d2	2	0	1	1	1	1

How does TF-IDF work?

TF-IDF

Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “The cat is on the table”

$$\text{TF}(t, d) = \text{raw count of } t \text{ in } d$$

	The	dog	is	on	table	cat
Term Frequency d1	2	1	1	1	1	0
Term Frequency d2	2	0	1	1	1	1
Documents with word	2	1	2	2	2	1

How does TF-IDF work?

TF-IDF

Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “The cat is on the table”

$$\text{IDF}(t, D) = \log\left(\frac{|D|}{n_t}\right)$$

	The	dog	is	on	table	cat
Term Frequency d1	2	1	1	1	1	0
Term Frequency d2	2	0	1	1	1	1
Documents with word (n_t)	2	1	2	2	2	1
IDF ($D = 2$)	0	0.3	0	0	0	0.3

How does TF-IDF work?

TF-IDF

Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “The cat is on the table”

$$\text{TF-IDF} = \text{TF}(t, d) * \text{IDF}(t, D)$$

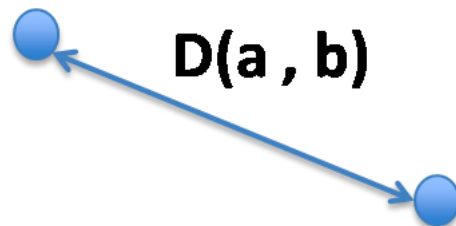
	The	dog	is	on	table	cat
Term Frequency d1	2	1	1	1	1	0
Term Frequency d2	2	0	1	1	1	1
Documents with word	2	1	2	2	2	1
IDF ($ D = 2$)	0	0.3	0	0	0	0.3
TF-IDF d1	0	0.3	0	0	0	0
TF-IDF d2	0	0	0	0	0	0.3



03.

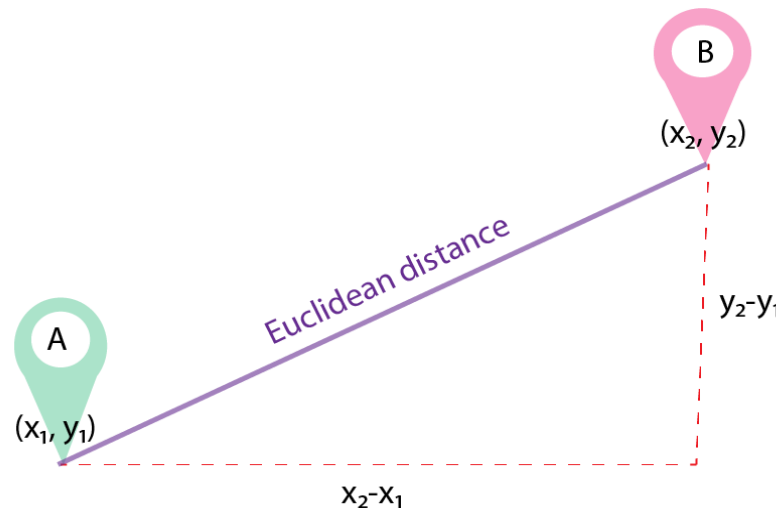
Distance Metrics

- Given 2 documents **we can transform them into feature space and compare them with simple distance metrics.**
- The feature space can be binary values or raw counts or TF-IDF of any kind of N-grams.
- If they **share a lot of features, they will be close to each other.**



The **Euclidean Distance** is the most common use of distance. When **data is dense** or **continuous**, this is the best proximity measure.

The Euclidean distance between two points is the **length of the path connecting them**. The Pythagorean theorem gives this distance between two points.



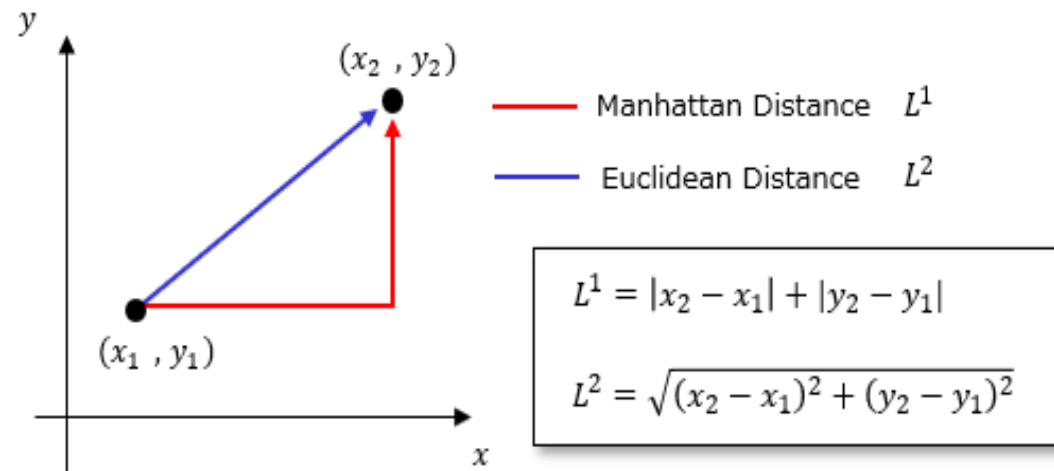
By representing documents as vectors, we can calculate, for example, the distance between documents...

	The	dog	is	on	table	cat
TF-IDF Doc 1	0	0.3	0	0	0	0
TF-IDF Doc 2	0	0	0	0	0	0.3

Euclidean:

$$D(d1,d2) = \sqrt{(0 - 0)^2 + (0.3 - 0)^2 + \dots} = \sqrt{(0.3)^2 + (-0.3)^2} = 0.42$$

Manhattan distance is a metric in which **the distance between two points is the sum of the absolute differences of their Cartesian coordinates**. In a simple way of saying it is the total sum of the difference between the x-coordinates and y-coordinates.



https://take2.com/N102_en.html

Cosine distance finds the **normalized dot product of the two attributes**. By determining the cosine similarity, we would effectively try to find the **cosine of the angle between the two objects**. The cosine of 0° is 1, and it is less than 1 for any other angle.

One of the reasons for the popularity of cosine similarity is that it is **very efficient to evaluate, especially for sparse vectors**.

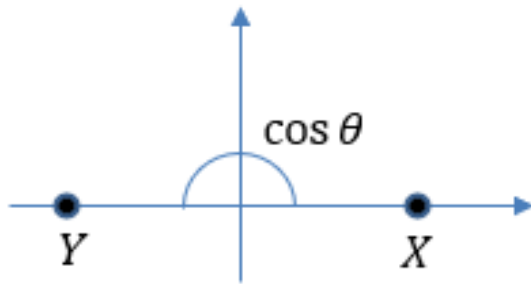
$$\text{similarity}(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}}$$

Cosine Distance

Distance Metrics

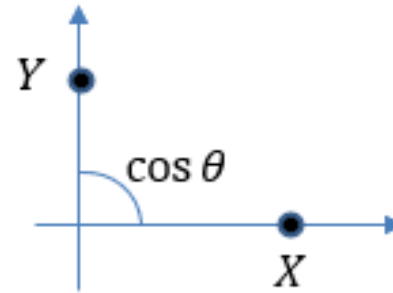
$$\cos \theta = r = -1$$

$\Rightarrow$ X and Y are
negatively correlated



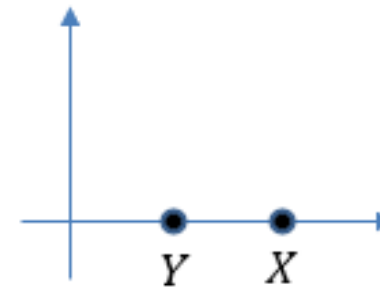
$$\cos \theta = r = 0$$

$\Rightarrow$ X and Y have
no correlation



$$\cos \theta = r = 1$$

$\Rightarrow$ X and Y are correlated



https://taketake2.com/N102_en.html

	The	dog	is	on	table	cat
TF-IDF Doc 1	0	0.3	0	0	0	0
TF-IDF Doc 2	0	0	0	0	0	0.3

Cosine Distance:

$$\text{Cos}(\theta) = \frac{v1 \cdot v2}{\|v1\| \|v2\|} = \frac{0}{0.3 * 0.3} = \mathbf{0} \text{ (90}^\circ\text{)}$$

$$v1 \cdot v2 = (0 * 0) + (0.3 * 0) + (0 * 0) + (0 * 0) + (0 * 0) + (0 * 0.3)$$

$$\|v1\| = \sqrt{0^2 + 0.3^2 + 0^2 + \dots} = \sqrt{0.3^2} = \sqrt{0.09} = 0.3$$

$$\|v2\| = \sqrt{0^2 + 0^2 + \dots + 0.3^2} = 0.3$$

2 ways to calculate distance through phonetics:

Phonetics – words

Orthography – words and documents

Soundex is a phonetic algorithm for indexing names by sound, as pronounced in English.

Example:

Smith -> S530

Smythe -> S530

George -> G620

Garage -> G620

Minimum Edit Distance (Levenshtein) counts the minimum number of operations required to transform one string into the other.

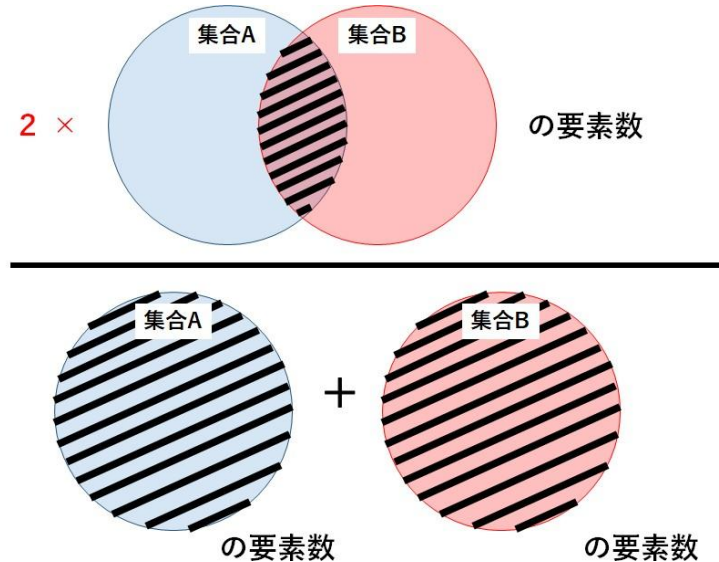
- **Insertion** of a single symbol. If $a = uv$, then inserting the symbol x produces uxv . This can also be denoted $\varepsilon \rightarrow x$, using ε to denote the empty string.
- **Deletion** of a single symbol changes uxv to uv ($x \rightarrow \varepsilon$).
- **Substitution** of a single symbol x for a symbol $y \neq x$ changes uxv to uyv ($x \rightarrow y$)

Jaccard coefficient measures similarity between finite sample sets, and is defined as the size of the **intersection** divided by the size of the **union** of the sample sets:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Sørensen–Dice is similar to Jaccard but **strings with different lengths are not so strongly penalized**

$$Dice(s, t) = 2 \times \frac{|s \cap t|}{|s| + |t|}$$

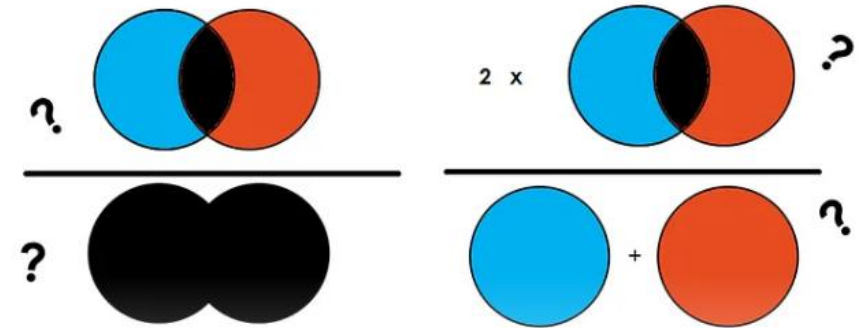
$$DSC(A, B) = \frac{2 \times \begin{array}{c} \text{集合A} \\ \text{集合B} \\ \text{の要素数} \end{array}}{\begin{array}{c} \text{集合A} \\ \text{の要素数} \end{array} + \begin{array}{c} \text{集合B} \\ \text{の要素数} \end{array}}$$
A Venn diagram illustrating the Dice Similarity formula. The numerator shows two overlapping circles, '集合A' (Set A) in light blue and '集合B' (Set B) in light red. The intersection of the two circles is shaded with black diagonal lines. To the left of the intersection is a red '2' followed by a red 'x'. To the right of the circles is the text 'の要素数' (number of elements). The denominator shows two separate circles, '集合A' and '集合B', both filled with black diagonal lines. A plus sign is placed between these two circles. Below each circle is the text 'の要素数'.

Dice vs Jaccard

Distance Metrics

Doc 1 (d1) – “The dog is on the table”

Doc 2 (d2) – “The cat is on the table”



Jaccard:

$d1 \cap d2 = \{\text{the, is, on, table}\} = 4$

$d1 \cup d2 = \{\text{The, dog, cat, is, on, table}\} = 6$

$$J(d1, d2) = 4/6 = 0.67$$

Dice:

$d1 \cap d2 = \{\text{The, is, on, table}\} = 4$

$d1 = \{\text{The, dog, is, on, table}\} = 5$

$d2 = \{\text{The, cat, is, on, table}\} = 5$

$$DSC(d1, d2) = 2*4/(5+5) = 8/10 = 0.8$$



Bibliographic references:

Dan Jurafsky and James H. Martin. **Speech and Language Processing** 3rd ed.

Chapter 3

<https://web.stanford.edu/~jurafsky/slp3/>

Thank you!

Acreditações e Certificações da NOVA IMS



Computing
Accreditation
Commission



Cofinanciado por

